Lab Sheet 5: Architecture, Patterns & Reuse

Objectives

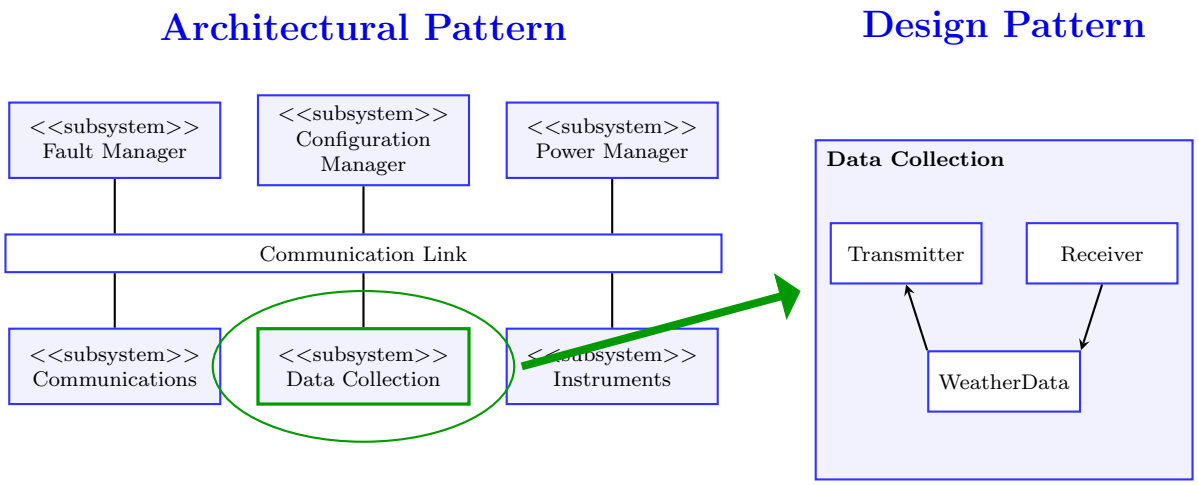
By completing today's workshop, you will:

- Define the high-level **Architectural Pattern** for your Ground Control Station.
- Apply a **Design Pattern** (Singleton or Facade) to your Robot API client code.
- Perform an **Open-Source License Audit** on your dependencies to ensure legal compliance.
- Define the **Provides and Requires Interfaces** for a core system component using CBSE principles.

Introduction: Bridging Design and Implementation

You have gathered your requirements, mapped your models, and set up your Agile board. Now, you are transitioning into implementation. Before you start writing thousands of lines of code, you must decide *how* that code will be organized (Architecture) and *how* you will solve recurring coding problems (Design Patterns). Because you are building this system using modern frameworks, you will also heavily rely on Software Reuse (Open Source).

Example:



OOP Refresher for Beginners

If you are new to programming, "Object-Oriented Programming" (OOP) might sound intimidating. However, it is just a way to organize your code so it is easier to read and reuse.

- **Class (The Blueprint):** A class is like an architectural blueprint. It defines what something is and what it can do. (e.g., `class RobotClient`).
- **Object / Instance (The House):** When you actually build a house from the blueprint, that is an "Object" or an "Instance." You can only have one blueprint, but you can build many houses from it.
- **Attributes (The Data):** Variables stored inside the class (e.g., `api_url = "http://localhost:5000"`).
- **Methods (The Actions):** Functions written inside the class (e.g., `def moveRobot(x, y):`).

If you need a deeper dive into these concepts before starting Task 2, please check the "Beginner OOP Resources" section at the end of this document!

Task 1: Mapping Your Architectural Pattern

Goal: Document the high-level organization of your software system.

The Concept: An architectural pattern provides a stylized description of good design practice. Whether you are using a Python/Flask backend with HTML templates, or a Node.js API with a React frontend, your system will likely follow a **Layered Architecture** or the **Model-View-Controller (MVC)** pattern.

Action:

1. In your GitHub repository, create or open your `ARCHITECTURE.md` file.
2. Write a brief section (1-2 paragraphs) declaring your chosen tech stack and mapping it to an architectural pattern.
3. Answer the following questions based on your specific implementation:
   - **If using MVC:** What framework handles the *View* (renders the UI)? What handles the *Controller* (processes HTTP requests and logic)? What acts as the *Model* (database/business logic)?
   - **If using Layered:** What libraries compose your Data Access layer? What comprises your User Interface layer?
4. *Tip: Use Mermaid.js from last week to sketch a quick diagram of these layers/components!*

Task 2: Applying a Design Pattern in Code

Goal: Use a low-level design pattern to solve a software construction problem.

The Concept: Your dashboard must communicate with the Virtual Robot via a REST API. You should not sprinkle raw HTTP requests (like `fetch` or `requests.post`) randomly throughout your codebase. Instead, you should create a dedicated "Robot API Client" class.

- **The Facade Pattern:** This class acts as a Facade, hiding the messy HTTP headers, JSON parsing, and error handling behind clean methods like `moveRobot(x, y)`.
- **The Singleton Pattern:** You only want *one* instance of this API client managing the connection across your entire app.

Action:

1. In your codebase, create a new file for your Robot API Client (e.g., `RobotClient.py`, `robotService.js`, `RobotAPI.java`, depending on your language).
2. Research how to implement the **Singleton Pattern** in your chosen language.
3. Write the "stub" (the class definition and empty methods) for this client. Ensure it implements the Singleton pattern so it cannot be instantiated more than once.
4. Add placeholder methods for the Facade operations: `getStatus()`, `move(x, y)`, and `reset()`.

Task 3: Open-Source License Audit (LEPSI)

Goal: Practice safe software reuse by evaluating the legal constraints of your dependencies.

The Concept: The developer of open-source code still legally owns it and places restrictions on how it is used via licenses. Permissive licenses (MIT, Apache) allow you to do almost anything, while Copyleft licenses (GPL) force your project to adopt the same open-source rules. You must verify you are not violating these terms.

Action:

1. Identify 3 to 5 third-party libraries/frameworks you plan to reuse in your project (e.g., React, Express, Flask, SQLite, Axios, Bootstrap).
2. Visit their official GitHub repositories or websites to find their `LICENSE` file.
3. Create a table in your documentation (or draft it for your final report) listing:
    - Component Name
    - Purpose in your system
    - License Type (e.g., MIT, Apache 2.0, GPLv3)
    - Is it Permissive or Copyleft?
4. Write a one-sentence conclusion stating whether your current combination of licenses imposes any legal restrictions on your Ground Control Station.

Task 4: CBSE Interface Specification

Goal: Define a system component as an independent service provider using Component-Based Software Engineering (CBSE) principles.

The Concept: In CBSE, components are abstract, stand-alone entities specified entirely by their interfaces. The **Provides interface** defines the services the component offers, while the **Requires interface** defines the external services it needs to execute.

Action:

1. Your assessment requires a **Mission Logger** to create an audit trail of user commands. Let's treat this as an independent CBSE component.
2. Open a markdown file or a diagramming tool.
3. Define the **Provides Interface** for the Mission Logger. What methods/services will it expose to the rest of your backend? (e.g., `logCommand(user, action)`, `exportLogs()`).
4. Define the **Requires Interface**. What external services does the Logger need to function properly? (e.g., `Database Connection`, `System Clock/Timestamp Service`).
5. *Stretch:* Sketch this out using the UML "ball and socket" notation demonstrated in the lecture.

Further Resources

Beginner OOP Resources:

- **Python OOP for Beginners (Real Python):** realpython.com/python3-object-oriented-programming/ - *A fantastic, easy-to-read guide on how to create classes and objects in Python.*
- **JavaScript Classes (MDN Web Docs):** developer.mozilla.org/.../Classes_in_JavaScript - *The official beginner's guide to writing OOP in modern JavaScript/Node.js.*

Architecture & Design Patterns:

- **Refactoring.Guru (Design Patterns):** https://refactoring.guru/design-patterns - *Excellent resource with code examples for Singletons and Facades in almost every language.*
- **ChooseALicense.com:** https://choosealicense.com/ - *A simple guide to understanding the legal implications of Open Source licenses.*
- **MVC Pattern Explained:** MDN Web Docs: MVC